

CLAUDE CODE — NOVIDADES DA PLATAFORMA

Comando /goal

Trabalho autônomo até uma condição de sucesso

Resumo técnico sobre sintaxe, funcionamento interno, comparação com outros modos, limitações e disponibilidade do comando /goal.

01 O que é o comando /goal

O /goal define uma condição de conclusão e instrui o Claude a continuar trabalhando autonomamente em direção a essa condição, sem exigir que você intervenha a cada etapa. Depois de cada turno, um modelo pequeno e rápido (o avaliador) verifica se a condição já foi atingida. Se ainda não, o Claude inicia outro turno automaticamente em vez de devolver o controle para você.

Principais casos de uso:

- Trabalho substancial com um estado final verificável.
- Tarefas que exigem vários turnos para concluir (ex.: migrar um módulo para uma nova API até todos os pontos de chamada compilarem e os testes passarem).
- Implementar um design doc até que todos os critérios de aceite sejam atendidos.
- Dividir arquivos grandes em módulos focados até cada um respeitar um limite de tamanho.
- Trabalhar em uma fila de issues rotuladas até a fila esvaziar.

02 Sintaxe e uso

Definir um objetivo:

```
/goal <condição>

# exemplo:
/goal all tests in test/auth pass and the lint step is clean
```

Consultar o objetivo ativo (sem argumentos): /goal · **Limpar um objetivo ativo**: /goal clear

```
# aliases aceitos: stop, off, reset, none, cancel

# uso não interativo:
claude -p "/goal <condição>"
claude -p "/goal <condição>" --output-format stream-json --verbose
```

Limite de tamanho da condição: até **4.000 caracteres**.

03 Como funciona internamente

- **1. Ativação** — rodar `/goal <condição>` inicia um turno imediatamente, usando a condição como diretiva.
- **2. Loop de avaliação** — ao final de cada turno, o Claude Code envia a condição e toda a conversa até então para o modelo pequeno e rápido configurado (padrão: Haiku). O avaliador devolve um veredito com justificativa: *ainda não atingida* (continua, usando a justificativa como orientação), *atingida* (encerra e registra "achieved"), ou *impossível* (encerra e marca "failed").
- **3. Condições de parada** — condição atingida; condição julgada impossível; erro que precisa correção manual; você roda `/goal clear`; ou outros erros entram em nova tentativa/pausam (ex.: rate limits).
- **4. Trabalho em segundo plano** — se um subagente ou comando em background ainda roda quando um turno termina, a avaliação é adiada; check-ins ocorrem a cada 30 min (configurável).
- **5. Retomada de sessão** — ao retomar uma sessão com objetivo ativo, o Claude Code restaura a condição, mas reinicia contagem de turnos, cronômetro e linha de base de tokens.

04 Exemplos reais da documentação

```
/goal all tests in test/auth pass and the lint step is clean
```

```
/goal CHANGELOG.md has an entry for every PR merged this week
```

```
/goal every call site to the old API has been migrated to the  
new one, all tests pass, and no references to the old module  
remain in the codebase
```

```
# com limite de turnos:
```

```
/goal implement user authentication or stop after 20 turns
```

05

Diferenças em relação a outros modos

RECURSO	/GOAL	/LOOP	STOP HOOK	AUTO MODE SOZINHO
Próximo turno começa quando	Turno anterior termina (ou check-in)	Intervalo de tempo passa	Turno anterior termina	Nunca (só aprova 1 turno)
Para quando	Condição atingida/impossível, limpeza manual, ou erro	Você para manualmente	Seu script/prompt decide	Claude julga o trabalho concluído
Escopo	Só a sessão atual	Só a sessão atual	Toda a sessão (no escopo)	—
Avaliação	Baseada em modelo (avaliador dedicado)	N/A	Baseada em modelo ou script	N/A

Diferença-chave em relação a um prompt comum: um prompt normal pede para o Claude fazer algo uma vez. O /goal envolve a condição em um Stop hook baseado em prompt, restrito à sessão, fazendo o Claude checar após cada turno se deve continuar ou parar — independente do próprio julgamento do Claude.

Diferença-chave em relação ao Auto Mode: o Auto Mode remove prompts de aprovação por ferramenta dentro de um único turno. O /goal remove prompts por turno, adicionando um avaliador separado que decide a conclusão com base na sua condição.

06

Limitações e configurações

- Um objetivo por sessão — definir um novo objetivo substitui o anterior.
- O avaliador não roda comandos nem lê arquivos por conta própria — só julga o que o Claude já expôs na conversa.
- A condição precisa ser verificável a partir da saída do Claude na transcrição.
- Sem progresso por vários turnos → o Claude Code interrompe o loop e devolve o controle.

VARIÁVEL	FINALIDADE	PADRÃO
ANTHROPIC_DEFAULT_HAIKU_MODEL	Troca o modelo usado como avaliador	Haiku (varia por provedor)
CLAUDE_CODE_GOAL_CHECKIN_MINUTES	Intervalo de check-in para trabalho em segundo plano longo	30 min; 0 desativa

07 Permissões e confiança de workspace

- O `/goal` segue a mesma regra de confiança de pasta de trabalho aplicada aos hooks.
- Fica indisponível se `disableAllHooks` estiver `true`, ou `allowManagedHooksOnly` nas configurações gerenciadas.
- Modo não interativo em pasta não confiável exige aprovação interativa antes — senão a execução é recusada.
- Não muda o modo de permissão: no Auto Mode os turnos rodam sem supervisão; no modo manual, o Claude segue pedindo aprovação para chamadas fora das configurações.

08 Disponibilidade por plataforma e versão

- Plataformas: CLI, app Desktop, extensão VS Code, Remote Control e modo não interativo (`-p`).
- Requer confiança da pasta de trabalho e um modelo pequeno/rápido configurado para avaliação.
- Disponível na API da Anthropic (padrão: Haiku) e em provedores terceiros (Bedrock, Vertex AI etc.).
- Sem restrição de plano — disponível para todos os usuários. Versão mínima: v2.1.234.
- Check-ins de espera: v2.1.236+ · Retries automáticos e avisos: v2.1.269+ · Restauração de sessão: v2.1.239+.

Principais pontos

- ✓ `/goal` define uma condição de sucesso e faz o Claude continuar trabalhando sozinho até atingi-la.
- ✓ Um avaliador (modelo pequeno e rápido) checa a condição após cada turno — atingida, impossível, ou segue em frente.
- ✓ Sintaxe: `/goal <condição>` para definir, `/goal` para consultar, `/goal clear` para encerrar.
- ✓ Combina bem com o Auto Mode para execução totalmente autônoma, sem prompts por turno.
- ✓ Uma sessão mantém um objetivo por vez; disponível para todos os planos, sem custo adicional.

Fonte oficial: code.claude.com/docs/en/goal.md — documentação relacionada: [scheduled-tasks.md \(/loop\)](#), [hooks-guide.md](#), [env-vars.md](#) e [model-config.md](#).